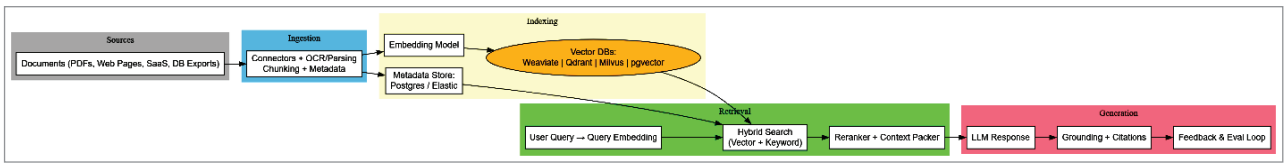

Figure 1: Enterprise RAG architecture

context. The effectiveness of the entire RAG system is directly tied to the quality of this step, as poor chunking leads to irrelevant retrieval and, consequently, a poor final response.

Embedding generation: Once the documents are chunked, each text chunk is passed through an embedding model. This model converts the text into a dense numerical array called a 'vector embedding'. These embeddings are designed to encode the semantic meaning of the text, such that chunks with similar meaning or context will have vectors that are mathematically 'close' to one another in a high-dimensional space.

Vector storage and indexing: The newly generated vector embeddings, along with their associated metadata, are then stored and indexed in the chosen vector database. This indexing process is what enables the database to perform a lightning-fast approximate nearest neighbour (ANN) search, quickly finding the most relevant vectors from a massive dataset.

Retrieval: This is the 'R' in RAG. When a user submits a query, it undergoes the same embedding process. The resulting query vector is then used to perform a similarity search in the vector database. The system retrieves the top 'k' most relevant chunks—those whose vectors are mathematically closest to the query vector—and pulls them from the database.

Augmented generation: The 'G' in RAG. The final stage involves passing the user's original query and the newly retrieved, relevant document chunks together to the large language model. The LLM uses this 'augmented context' to generate a response that is not only coherent and well-written but also factually grounded in the provided source material.

Practical guide: Building an RAG system on Linux

For the open source enthusiast, a fully self-hosted, Linux-native RAG system is not a distant vision but a practical reality. This guide provides a hands-on blueprint using Qdrant and a Python-based orchestration framework to create a functional prototype. This will be the first part in the overall series. In the coming parts, I will try to cover the rest of the solutions mentioned in the above section.

To build a RAG system that acts as a knowledgeable AI assistant for your enterprise documents, we will walk through a complete, self-hosted implementation using Python, an open source vector database like Qdrant, and a FOSS embedding model.

Before stepping into the journey of implementation, let us quickly comprehend the basics of the product. This should help you understand each step better and configure accordingly.

Term	Meaning
Collections	Collections define a named set of points that you can use for your search.
Payload	A payload describes information that you can store with vectors.
Points	Points are a record which consists of a vector and an optional payload.
Search	Search describes similarity search, which sets up related objects close to each other in vector space.
Explore	Explore includes several APIs for exploring data in your collections.
Hybrid queries	Hybrid queries combine multiple queries or perform them in more than one stage.
Filtering	Filtering defines various database-style clauses, conditions, and more.
Optimizer	Optimizers are options to rebuild database structures for faster search. These include a vacuum, a merge, and an indexing optimizer.
Storage	Storage is the configuration of storage in segments, which include indexes and an ID mapper.
Indexing	Indexing lists and describes available indexes. These include payload, vector, sparse vector, and a filterable index.
Snapshots	Snapshots describe the backup/restore process (and more) for each node at specific times.

Step 1: Setting up the environment

Before you begin, you need to ensure your Linux system has the necessary tools. This includes Python .8+ and Docker. You will also need to install the required Python libraries.

First download the latest Qdrant image from the Docker Hub:

```
docker pull qdrant/qdrant
```

Then run the service:

```
docker run -p 6333:6333 -p 6334:6334 \
-v "$(pwd) /qdrant_storage: /qdrant/storage:z" \
qdrant/qdrant
```